

GearGuard: A Detector-Agnostic Mobile Architecture for Personal Protective Equipment Compliance Monitoring

Author Name

Department, Institution · City, Country · email@example.com

Abstract—Verifying that workers wear the required personal protective equipment (PPE) before entering a work area is a routine but error-prone safety task. Existing research concentrates on the detection model, while the surrounding workflow—identifying the worker, applying site-specific rules, recording the result and producing audit evidence—receives little attention. This paper presents GearGuard, a cross-platform mobile application covering that workflow end to end. We contribute (i) a formal, detector-independent definition of per-scan and per-worker compliance, (ii) a layered architecture in which a single interface isolates the machine-learning model from the user interface, storage and reporting, and (iii) an open-source Flutter reference implementation with scan, attendance, ranking, archive and PDF-report features, verified by unit and end-to-end widget tests. **[TO BE COMPLETED: one sentence with the headline result of the detector benchmark and usability study.]**

Keywords—personal protective equipment, occupational safety, mobile application, object detection, software architecture, Flutter

I. INTRODUCTION

Construction, manufacturing and logistics sites require workers to wear helmets, high-visibility vests, gloves and safety footwear. Compliance is usually checked by a supervisor at the gate, which is slow, inconsistent and leaves no structured record. Vision-based PPE detection has been studied extensively [2], building on real-time detectors such as YOLO [1]. A detector alone, however, is not a usable safety tool: supervisors must also know *who* was scanned, *which* rules applied, and be able to retrieve history for audits.

Contributions: (1) a detector-independent formalisation of compliance; (2) a layered architecture treating the detector as a replaceable component; (3) an open-source implementation and its verification.

[TO BE COMPLETED: 3–5 sentences positioning the work against recent literature; verify every citation.]

II. RELATED WORK

[TO BE COMPLETED: survey of (a) deep-learning PPE detection on construction and industrial sites, (b) mobile/edge deployment of detectors, (c) digital safety-management and attendance systems. Peer-reviewed sources only; state the gap: an end-to-end workflow with a swappable detector.]

III. PROBLEM FORMULATION

Let P be the set of supported PPE types (helmet, vest, gloves, boots, goggles, mask). A site fixes a required subset $R \subseteq P$, $R \neq \emptyset$, and a confidence threshold $\tau \in [0,1]$. For a frame, the detector returns a confidence $c_p \in [0,1]$ for every $p \in P$. An item is *detected* when $c_p \geq \tau$. The set of missing items is

$$M = \{p \in R \mid c_p < \tau\}. \quad (1)$$

A scan is *compliant* iff $M = \emptyset$, and its *score* is

$$s = (|R| - |M|) / |R|. \quad (2)$$

For a worker w with scan set S_w , the compliance rate is

$$C_w = |\{x \in S_w \mid M_x = \emptyset\}| / |S_w|, \quad (3)$$

and workers are ranked by C_w in descending order, ties broken by the mean score. The daily site rate is the same ratio over all scans of that day. R and τ are stored with each scan, so historical results stay valid when site rules change.

IV. SYSTEM DESIGN

A. Layered architecture

GearGuard has four layers: *domain* (pure Dart value types), *data* (detector, storage, PDF builder), *state* (flutter_bloc cubits with immutable states) and *presentation* (feature modules). Derived quantities— C_w , rankings, most-missed equipment—are computed in the state layer, not in widgets, which keeps them unit-testable.

presentation	dashboard scanner attendance
	team archive settings
state	SafetyCubit SettingsCubit
data	PpeDetector SafetyRepository
report_pdf	
domain	PpeItem Employee ScanRecord

B. Detector interface

```
abstract interface class PpeDetector {
  Future<Map<PpeItem, double>> detect();
}
```

This is the only contract between the model and the rest of the app. It is injected at the application root, so replacing the simulated detector with an on-device model (e.g., a YOLO network through a platform channel) needs no change to any screen, cubit or report.

C. Interaction design

Bottom navigation exposes the five daily tasks (Home, Attendance, Scan, Team, Archive). Violations are sorted first in the attendance list; workers who have not checked in have a one-tap “Scan” action; the scan result shows a per-item confidence checklist with a bounding-box overlay (Fig. 1);

status is never conveyed by colour alone. Light and dark themes are provided.

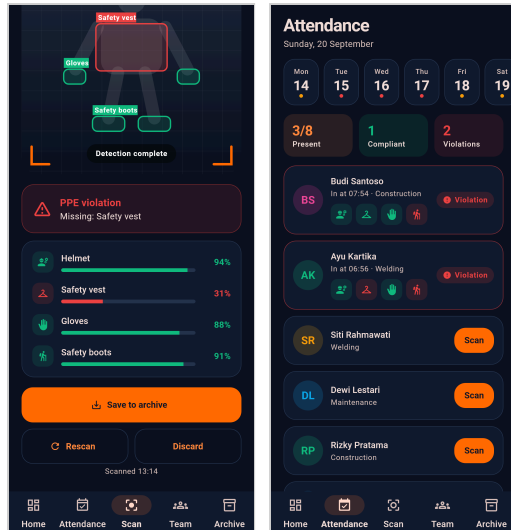


Fig. 1. Scan result (left) and attendance view (right).

D. Persistence and reporting

Employees and scans are stored as versioned JSON in platform key-value storage (mobile, desktop and web). A generator produces a branded A4 PDF with summary indicators and a per-scan table for one scan, one worker or the current archive filter.

V. IMPLEMENTATION AND VERIFICATION

The implementation targets Flutter 3.41 / Dart 3.11 with 32 source files ($\approx 4.9k$ lines) and Android, iOS, web, Windows and macOS runners. It passes `flutter analyze` with no errors or warnings under the `very_good_analysis` rule set (two informational lints remain). The suite has seven tests: three unit tests (compliance maths and JSON round-trip; deterministic seed data; detector output range) and four end-to-end widget tests (onboarding gate; running a scan and saving a violation; adding an employee; filtering the archive).

The current build uses a simulated detector that draws item confidences at random after a short delay. It exists to exercise the whole workflow without a camera or trained model; consequently *no detection accuracy is claimed in this paper*. The authors' earlier prototype (`ppe_apk_prototype`) ships a YOLO model; connecting it behind `PpeDetector` is the immediate next step.

VI. EVALUATION

[TO BE COMPLETED — minimum recommended content]

A. Detector benchmark: dataset, split, hyper-parameters, hardware; precision, recall, $mAP@0.5$ and $mAP@0.5:0.95$ per class; on-device latency (ms/frame) and memory on named phones.

B. Usability study: n participants; task success and time for scan / find violation / export report; System Usability Scale [3]; protocol and ethics statement.

C. Workflow comparison against a paper checklist baseline (time to record).

VII. LIMITATIONS

Compliance is decided per item from a single frame; occlusion, lighting and multiple workers per frame are not modelled. Identity is selected from a list rather than recognised. Storage is local to the device. Camera-based monitoring of people raises privacy and consent issues; deployments must define retention and consent policies and should store scores rather than images. **[TO BE COMPLETED: applicable regulation.]**

VIII. CONCLUSION

We presented GearGuard, a detector-agnostic architecture and open-source implementation of the PPE compliance workflow. Future work: a real on-device detector behind the same interface, a field usability study and department-level reporting.

REFERENCES

1. J. Redmon, S. Divvala, R. Girshick, A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE CVPR*, 2016, pp. 779–788.
2. N. D. Nath, A. H. Behzadan, S. G. Paal, "Deep learning for site safety: Real-time detection of personal protective equipment," *Automation in Construction*, vol. 112, 2020, Art. 103085.
3. J. Brooke, "SUS: A 'quick and dirty' usability scale," in *Usability Evaluation in Industry*, Taylor & Francis, 1996, pp. 189–194.

Verify every reference against the publisher page before submission.